AI-Powered Enterprise E-Commerce Platform

API Requirements Specification

Document Information

Item	Description
Project Name	AI-Powered Enterprise E-Commerce Platform
Document Type	API Requirements Specification
Version	1.0
Prepared By	Project Team
Status	Draft

Revision History

Version	Date	Description
1.0	DD/MM/YYYY	Initial Version

Table of Contents

---

1. Introduction

The API Requirements Specification defines all application programming interfaces used by the AI-Powered Enterprise E-Commerce Platform.

The APIs provide communication between:

- Frontend and Backend Services
- Spring Boot Services
- Python AI Services
- Third-Party Services

This document acts as the contract between different system components.

---

2. Purpose

The purpose of this document is to:

- define service interfaces,
- standardize communication,
- improve maintainability,
- support scalability,
- reduce integration risks.

---

3. API Objectives

The APIs should:

API-01

Provide secure communication.

API-02

Support service independence.

API-03

Maintain consistent response structures.

API-04

Support future versioning.

API-05

Enable AI service integration.

---

4. API Architecture

High-Level Architecture

```
Frontend
     ↓
API Gateway
     ↓
-----------------------------------
| Spring Boot Services            |
| FastAPI AI Services             |
-----------------------------------
     ↓
Database / Cache / Queue
```

Communication Types

Synchronous

- REST APIs
- HTTPS

Asynchronous

- RabbitMQ Events

---

5. API Standards

Protocol

HTTPS

Data Format

JSON

Character Encoding

UTF-8

Naming Convention

```
/api/v1/resource-name
```

Example:

```
/api/v1/products
```

HTTP Methods

Method	Purpose
GET	Retrieve Data
POST	Create Data
PUT	Update Data
PATCH	Partial Update
DELETE	Remove Data

6. Authentication APIs

Base URL:

```
/api/v1/auth
```

Register User

Endpoint

```
POST /register
```

Request:

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john@email.com",
  "password": "********"
}
```

Response:

```
{
  "message": "User registered successfully"
}
```

Login

Endpoint

```
POST /login
```

Response:

```
{
  "accessToken": "JWT_TOKEN",
  "refreshToken": "REFRESH_TOKEN"
}
```

Logout

```
POST /logout
```

Refresh Token

```
POST /refresh-token
```

---

7. User APIs

Base URL:

```
/api/v1/users
```

Get Profile

```
GET /me
```

Update Profile

```
PUT /me
```

Get User Orders

```
GET /me/orders
```

---

8. Product APIs

Base URL:

```
/api/v1/products
```

Get All Products

```
GET /
```

Query Parameters:

```
?page=
&size=
&sort=
```

Search Products

```
GET /search
```

Example:

```
/search?keyword=laptop
```

Get Product By ID

```
GET /{productId}
```

Create Product

```
POST /
```

Update Product

```
PUT /{productId}
```

Delete Product

```
DELETE /{productId}
```

Product Reviews

```
GET /{productId}/reviews
POST /{productId}/reviews
```

---

9. Category APIs

```
/api/v1/categories
```

Endpoints:

```
GET /
POST /
PUT /{id}
DELETE /{id}
```

---

10. Cart APIs

Base URL:

```
/api/v1/cart
```

Get Cart

```
GET /
```

Add Item

```
POST /items
```

Update Quantity

```
PUT /items/{itemId}
```

Remove Item

```
DELETE /items/{itemId}
```

Clear Cart

```
DELETE /
```

---

11. Wishlist APIs

```
/api/v1/wishlist
```

Endpoints:

```
GET /
POST /{productId}
DELETE /{productId}
```

---

12. Order APIs

Base URL:

```
/api/v1/orders
```

Create Order

```
POST /
```

Get Order History

```
GET /
```

Get Order By ID

```
GET /{orderId}
```

Cancel Order

```
PATCH /{orderId}/cancel
```

Track Order

```
GET /{orderId}/tracking
```

13. Payment APIs

Base URL:

```
/api/v1/payments
```

Initiate Payment

```
POST /
```

Payment Callback

```
POST /callback
```

Payment Status

```
GET /{paymentId}
```

---

14. Inventory APIs

Base URL:

```
/api/v1/inventory
```

Get Inventory

```
GET /
```

Update Stock

```
PATCH /{productId}
```

Low Stock Products

```
GET /low-stock
```

---

15. Recommendation APIs

Base URL:

```
/api/v1/recommendations
```

Service:

FastAPI

Personalized Recommendations

```
GET /users/{userId}
```

Response:

```
{
  "products": [
    1,
    5,
    10
  ]
}
```

Similar Products

```
GET /products/{productId}
```

Trending Products

```
GET /trending
```

---

16. Fraud Detection APIs

Base URL:

```
/api/v1/fraud
```

Service:

FastAPI

Fraud Analysis

```
POST /analyze
```

Request:

```
{
  "orderId": 1001
}
```

Response:

```
{
  "riskScore": 0.85,
  "status": "HIGH_RISK"
}
```

---

17. Forecasting APIs

Base URL:

```
/api/v1/forecast
```

Product Demand Forecast

```
GET /products/{productId}
```

Sales Forecast

```
GET /sales
```

---

18. Customer Segmentation APIs

```
/api/v1/segments
```

Endpoints:

```
GET /customers
GET /summary
```

---

19. Analytics APIs

Base URL:

```
/api/v1/analytics
```

Dashboard Summary

```
GET /dashboard
```

Revenue Reports

```
GET /revenue
```

Product Reports

```
GET /products
```

Customer Reports

```
GET /customers
```

---

20. Notification APIs

Base URL:

```
/api/v1/notifications
```

Get Notifications

```
GET /
```

Mark As Read

```
PATCH /{notificationId}/read
```

---

21. Internal Service APIs

Spring Boot → Recommendation Service

```
POST /internal/recommendations
```

Spring Boot → Fraud Service

```
POST /internal/fraud-check
```

Spring Boot → Forecast Service

```
POST /internal/forecast
```

---

22. Event-Driven APIs

Events published through RabbitMQ.

Order Created Event

```
{
  "event": "ORDER_CREATED",
  "orderId": 1001
}
```

Payment Completed Event

```
{
  "event": "PAYMENT_COMPLETED",
  "paymentId": 2001
}
```

Inventory Updated Event

```
{
  "event": "INVENTORY_UPDATED"
}
```

---

23. Standard Response Format

Success Response

```
{
  "success": true,
  "message": "Operation completed",
```

```
  "data": {}
}
```

Error Response

```
{
  "success": false,
  "message": "Validation failed",
  "errors": []
}
```

---

24. HTTP Status Codes

Code	Meaning
200	OK
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
422	Validation Error
500	Internal Server Error

---

25. Security Requirements

Authentication

JWT Authentication.

Authorization

Role-Based Access Control.

Transport Security

HTTPS/TLS.

Rate Limiting

Protect against abuse.

API Validation

Validate all requests.

API Logging

Log all critical operations.

---

26. Versioning Strategy

Versioning format:

```
/api/v1/
```

Future versions:

```
/api/v2/
/api/v3/
```

Backward compatibility should be maintained whenever possible.

---

27. API Documentation Requirements

Documentation technologies:

- OpenAPI
- Swagger UI

Every API must contain:

- Description
- Parameters
- Request examples

- Response examples
- Error responses

---

28. Performance Requirements

Average Response Time

< 2 seconds.

Recommendation APIs

< 5 seconds.

Concurrent Requests

Support:

1,000+ users.

---

29. Future Enhancements

Possible future improvements:

- GraphQL APIs
- gRPC communication
- Apache Kafka integration
- API Gateway Service Discovery
- WebSocket notifications
- Real-time recommendation APIs

---

30. Conclusion

The API architecture provides:

- service independence,
- scalability,
- maintainability,
- secure communication,
- and strong AI integration capabilities.

The API specification establishes a standardized communication contract for the entire enterprise platform.

References

[1] Richardson, L., & Amundsen, M. (2013). RESTful Web APIs.

[2] Fielding, R. (2000). Architectural Styles and the Design of Network-based Software Architectures.

[3] Newman, S. (2021). Building Microservices (2nd Edition).

[4] OpenAPI Specification Documentation.

[5] Swagger Documentation.

[6] OWASP API Security Top 10.

[7] Spring Boot Documentation.

[8] FastAPI Documentation.